



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

DEPARTAMENTO DE COMPUTACIÓN Y SIMULACIÓN DE SISTEMAS

LICENCIATURA EN INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

Asignatura:

Sistemas Operativos

Asignación:

Tarea #2: Análisis Comparativo de Estructuras de Sistemas Operativos

Profesor:

Prof. Isacio Tamayo

Estudiante:

James Tack

Grupo:

1IL143

Fecha de entrega:

3 de septiembre de 2026

2do Semestre 2026

Estructuras de sistemas operativos

La tarea pide comparar cinco formas de organizar un sistema operativo por dentro. No son cinco productos: son cinco decisiones sobre dónde va cada pieza y quién puede tocar el hardware. Casi siempre se paga con la misma moneda, velocidad contra aislamiento, y por eso ningún sistema serio de hoy usa una sola en estado puro.

Cuadro comparativo

Estructura	Cómo se organiza	Ventajas	Desventajas	Ejemplos
Monolítico	Un solo programa grande corriendo en modo núcleo. Procesos, memoria, archivos, red y drivers comparten el mismo espacio de direcciones y se llaman entre sí como funciones normales.	El más rápido de todos: una llamada interna no cambia de contexto ni copia mensajes. Acceso directo al hardware y código maduro después de décadas.	Sin fronteras internas: cualquier error escribe sobre memoria ajena y tumba la máquina completa. Crece hasta volverse difícil de leer y de probar.	UNIX clásico, MS-DOS, Linux (monolítico modular), FreeBSD
Por capas / niveles	Niveles numerados uno encima de otro. La capa 0 toca el hardware y cada capa solo puede pedirle servicios a la de abajo y ofrecérselos a la de arriba.	Se depura y se verifica capa por capa: si el nivel 1 ya está probado, el error del nivel 2 está en el nivel 2. Muy claro para enseñar y para documentar.	Decidir en qué nivel va cada cosa es un rompecabezas y hay dependencias que no se dejan ordenar. Cada frontera cuesta tiempo, así que una petición que baja cinco niveles paga cinco peajes.	THE (Dijkstra, 1968), MULTICS, OS/2
Microkernel	En modo núcleo queda lo mínimo: planificación, memoria y paso de mensajes. Drivers, sistema de archivos y pila de red salen a modo usuario como procesos servidores normales.	Un componente que falla se muere solo y se puede reiniciar sin apagar nada. El núcleo es tan pequeño que se puede auditar de verdad, y hasta verificar formalmente.	Todo servicio se pide por mensajes, y cada mensaje son dos cambios de modo y copias de datos. Ese sobrecosto fue el que hundió a los microkernels de primera generación.	MINIX 3, QNX, L4, Mach, GNU Hurd
Máquina virtual	Un hipervisor se sienta sobre el hardware real y fabrica varias copias virtuales de la máquina completa. Sobre cada copia se instala un sistema operativo entero, que no se entera.	Aislamiento fuerte entre huéspedes y varios sistemas distintos conviviendo en un mismo equipo. Snapshots, clonado y migración en caliente salen casi gratis.	Se pierde rendimiento en la traducción, sobre todo en entrada/salida. El hipervisor es un punto único de falla y los huéspedes se pelean por CPU, memoria y disco.	IBM VM/370, VMware ESXi, VirtualBox, Hyper-V, KVM, Xen
Cliente / servidor	El sistema se parte en procesos servidores que ofrecen un servicio cada uno y en clientes que se lo piden por mensajes. El núcleo casi solo transporta esos mensajes.	Al cliente le da igual si el servidor está en la misma máquina o al otro lado de la red, así que el modelo escala a sistemas distribuidos sin cambiar de idea.	Depende por completo del paso de mensajes: latencia, serialización y una red que puede caerse. Hay tareas que necesitan tocar el hardware directo y ahí el modelo no encaja.	Amoeba, Plan 9, X Window System, subsistemas de Windows NT

Preguntas de análisis

A. Un driver colapsa: monolítico contra microkernel

En el monolítico se cae todo. El driver no es un programa aparte que el sistema esté supervisando: está cargado dentro del mismo espacio de direcciones del núcleo y corriendo con privilegios de anillo 0. Cuando desreferencia un puntero nulo o escribe fuera de rango, no está estropeando su

propia memoria, está estropeando la del sistema entero. Y como el núcleo no tiene forma de saber qué tan corrompido quedó, hace lo único seguro que puede hacer: se detiene. Eso es la pantalla azul de Windows o el *kernel panic* de Linux. Se pierde lo que no estuviera guardado y hay que reiniciar.

En el microkernel se cae el driver y nada más. Ahí el driver es un proceso de usuario común, con su propio espacio de direcciones y sin permiso para tocar la memoria del núcleo. El acceso inválido lo bloquea la unidad de gestión de memoria y el proceso muere solo. El núcleo sigue vivo porque nunca estuvo en riesgo. MINIX 3 va un paso más allá con el servidor de reencarnación, que nota que el driver murió y arranca una copia limpia. Los programas que no usaban ese dispositivo ni se enteran; los que sí, ven un error de entrada/salida o una pausa.

Esa es toda la discusión, en el fondo. El monolítico apuesta a que sus drivers no fallan, porque si fallan no hay red debajo. El microkernel asume que van a fallar y arma el sistema alrededor de esa suposición. La segunda postura envejece mejor, sobre todo sabiendo que buena parte del código de un núcleo son drivers de terceros.

B. Por qué los sistemas modernos se llaman híbridos

Porque los dos extremos puros pierden en algo importante y nadie quiso pagar la cuenta completa. El monolítico gana velocidad y pierde confiabilidad; el microkernel, al revés. Windows NT y Linux se pararon en el medio, cada uno llegando desde un lado distinto.

Windows NT nació con planos de microkernel. Dave Cutler lo partió en un Ejecutivo de gestores independientes, una capa de abstracción de hardware y subsistemas de entorno (Win32, POSIX, OS/2) que corrían como procesos de usuario. Y luego llegó la factura de rendimiento: en NT 4.0 Microsoft movió el gestor de ventanas y el motor gráfico a modo núcleo, dentro de `win32k.sys`, porque el ida y vuelta de mensajes hacía que la interfaz se sintiera lenta. Hoy los drivers y los sistemas de archivos también viven en modo núcleo. Queda un diseño ordenado por dentro, pero ejecutándose casi todo en el mismo lugar.

Linux viene del lado contrario. Es monolítico sin disimulo, y Torvalds lo defendió así en aquella discusión de 1992 con Tanenbaum. Pero los módulos cargables le dieron algo que el monolítico clásico no tenía: con `insmod` y `rmmod` se mete o se saca un driver con el sistema encendido, sin recompilar ni reiniciar. La flexibilidad es de microkernel aunque el código siga compartiendo el mismo espacio de direcciones. Y con FUSE hasta se pueden escribir sistemas de archivos en modo usuario, que es justo la idea del microkernel.

Así que híbrido no es una estructura nueva ni un punto medio elegante. Es el nombre de una transacción: quedarse con la modularidad del microkernel donde no cuesta caro y con la velocidad del monolítico donde sí. Uno llegó ahí aflojando un diseño limpio; el otro, ordenando un diseño sucio.

Referencias

[1] A. S. Tanenbaum y H. Bos, Modern Operating Systems, 4a ed. Boston, MA, EE. UU.: Pearson, 2015, cap. 1 y 12.

[2] A. Silberschatz, P. B. Galvin y G. Gagne, Operating System Concepts, 10a ed. Hoboken, NJ, EE. UU.: Wiley, 2018, cap. 2.

[3] M. E. Russinovich, D. A. Solomon y A. Ionescu, Windows Internals, Part 1, 7a ed. Redmond, WA: Microsoft Press, 2017, cap. 2.

[4] J. N. Herder et al., "Fault isolation for device drivers", en Proc. IEEE/IFIP DSN, Estoril, Portugal, 2009, pp. 33-42.